Experiment No. 3

Implementation and Analysis of Spatial Filtering Techniques using Low-Pass and High-Pass Filters

Aim

To implement and analyze spatial domain filtering techniques using low-pass and high-pass filters for image smoothing, noise reduction, edge enhancement, and feature preservation using Python and OpenCV.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load image
img = cv2.imread("citystreet.jpg")

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# Display image
plt.figure(figsize=(8,5))
plt.imshow(img_rgb)
plt.title("Original City Street Image")
plt.axis("off")
plt.show()
```




```
# Convert RGB image to Grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Display grayscale image
plt.figure(figsize=(8,5))
plt.imshow(gray, cmap='gray')
plt.title("Original Grayscale Image")
```

```
plt.axis("off")
plt.show()

# Image properties
print("Image Shape:", gray.shape)
print("Minimum Pixel Value:", np.min(gray))
print("Maximum Pixel Value:", np.max(gray))
print("Mean Pixel Value:", np.mean(gray))
```



Original Image Analysis

The selected city street image contains buildings, roads, shadows, windows, and other structural elements with clearly visible edges and textures.

The image contains both smooth regions and high-detail regions, making it suitable for evaluating the effects of low-pass and high-pass filtering techniques.

The grayscale version preserves the structural information required for edge detection and filtering operations.

```
# Gaussian Blur
gaussian = cv2.GaussianBlur(gray, (5,5), 0)

# Display comparison
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(gaussian, cmap='gray')
plt.title("Gaussian Blur")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```



Gaussian Blur Observation

Gaussian Blur smooths the image by reducing high-frequency noise while preserving the overall structure of important objects.

The edges remain visible, but fine details and textures become slightly softer. This filter is widely used for noise reduction before further image processing tasks.

```
# Median Filter
median = cv2.medianBlur(gray, 5)

# Display comparison
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(median, cmap='gray')
plt.title("Median Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Median Filter Observation

The Median Filter replaces each pixel with the median value of its neighboring pixels.

It is particularly effective for removing impulse noise while preserving edges better than many averaging techniques.

The overall image appears smoother with important structures largely preserved.

```
# Average (Mean) Filter
average = cv2.blur(gray, (5,5))

# Display comparison
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(average, cmap='gray')
plt.title("Average Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Average Filter Observation

The Average Filter smooths the image by replacing each pixel with the average value of its neighboring pixels.

It reduces noise and minor intensity variations but may blur edges and fine details more than Gaussian filtering.

The resulting image appears smoother, but some structural details become less distinct.

```
plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(gaussian, cmap='gray')
plt.title("Gaussian Blur")
plt.axis("off")

plt.subplot(1,3,3)
plt.imshow(average, cmap='gray')
plt.title("Average Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Gaussian vs Average Filter Comparison

Both filters reduce image noise and smooth the image.

The Gaussian Filter assigns higher weights to pixels near the center and generally preserves image structures better.

The Average Filter treats all neighboring pixels equally, resulting in stronger blurring of edges and fine details.

Therefore, Gaussian filtering usually provides better smoothing while maintaining important image features.

```
# Laplacian Filter
laplacian = cv2.Laplacian(gray, cv2.CV_64F)

# Convert for display
laplacian_display = cv2.convertScaleAbs(laplacian)

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(laplacian_display, cmap='gray')
plt.title("Laplacian Filter")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Laplacian Filter Observation

The Laplacian Filter highlights regions with rapid intensity changes and enhances image edges.

Unlike smoothing filters, it emphasizes fine details and object boundaries. The resulting image contains strong edge information that can be useful for feature extraction and object detection.

```
# Sobel Horizontal (X)
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)

# Sobel Vertical (Y)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

# Convert for display
sobel_x_display = cv2.convertScaleAbs(sobel_x)
sobel_y_display = cv2.convertScaleAbs(sobel_y)

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Original Image")
plt.axis("off")

plt.subplot(1,3,2)
plt.imshow(sobel_x_display, cmap='gray')
plt.title("Sobel X")
plt.axis("off")

plt.subplot(1,3,3)
```

```
plt.imshow(sobel_y_display, cmap='gray')
plt.title("Sobel Y")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Sobel Edge Detection Observation

The Sobel operator calculates image gradients in specific directions.

Sobel X emphasizes vertical edges, while Sobel Y emphasizes horizontal edges.

These operators help identify object boundaries, structural features, and directional edge information useful in computer vision applications.

```
plt.figure(figsize=(18,10))

plt.subplot(2,3,1)
plt.imshow(gray, cmap='gray')
plt.title("Original")
plt.axis("off")

plt.subplot(2,3,2)
plt.imshow(gaussian, cmap='gray')
plt.title("Gaussian Blur")
plt.axis("off")

plt.subplot(2,3,3)
plt.imshow(median, cmap='gray')
plt.title("Median Filter")
plt.axis("off")

plt.subplot(2,3,4)
plt.imshow(average, cmap='gray')
plt.title("Average Filter")
plt.axis("off")

plt.subplot(2,3,5)
plt.imshow(laplacian_display, cmap='gray')
plt.title("Laplacian")
```

```
plt.axis("off")

plt.subplot(2,3,6)
plt.imshow(sobel_x_display, cmap='gray')
plt.title("Sobel X")
plt.axis("off")

plt.tight_layout()
plt.show()
```



Comparison of Spatial Filtering Techniques

Gaussian Blur effectively reduces noise while preserving important image structures.

Median Filtering provides good noise removal and preserves edges better than averaging techniques.

Average Filtering smooths the image but may blur important edges and fine details.

Laplacian Filtering enhances image edges and highlights abrupt intensity changes.

Sobel Filtering detects image gradients and provides directional edge information useful for object detection and feature extraction.

Low-pass filters are useful for noise reduction, whereas high-pass filters are useful for edge enhancement and detail extraction.

Applications of Spatial Filtering

1. Medical image enhancement and preprocessing.
2. Surveillance and security systems.
3. Autonomous vehicles and driver assistance systems.
4. Remote sensing and satellite image analysis.
5. Industrial inspection and quality control.
6. Object detection and recognition systems.

Conclusion

In this experiment, various spatial filtering techniques were implemented and analyzed using Python and OpenCV.

Low-pass filters such as Gaussian, Median, and Average filters successfully reduced image noise and smoothed image regions.

High-pass filters such as Laplacian and Sobel operators enhanced image edges and highlighted structural details.

The experiment demonstrated that different filtering techniques serve different purposes, and the choice of filter depends on the specific requirements of a computer vision application.

Theory Questions – Spatial Filtering in Digital Image Processing

1. What is spatial filtering? How is it used in digital image processing?

Spatial filtering is a technique used in digital image processing where the value of a pixel is modified based on the values of its neighboring pixels. It operates directly on the pixels of an image using a small matrix called a **filter, mask, or kernel**.

The kernel is moved across the image, and mathematical operations are performed between the kernel values and the corresponding image pixels. Spatial filtering is commonly used for **image smoothing, noise removal, sharpening, edge detection, and feature enhancement**.

---

2. Differentiate between Low-Pass Filters and High-Pass Filters with suitable examples.

Low-Pass Filter	High-Pass Filter
Allows low-frequency components to pass through.	Allows high-frequency components to pass through.
Reduces rapid intensity changes in an image.	Enhances rapid intensity changes.
Mainly used for smoothing and noise reduction.	Mainly used for sharpening and edge detection.
Makes the image smoother or slightly blurred.	Makes edges and fine details more prominent.
Examples: Average filter and Gaussian filter.	Examples: Laplacian filter and Sobel filter.

Example: A Gaussian filter can be used to reduce noise and smooth an image, while a Laplacian filter can be used to highlight edges.

---

3. Compare Average Filter, Gaussian Filter, and Median Filter based on their working principles and applications.

Filter	Working Principle	
Average Filter	Replaces each pixel with the average of its neighboring pixels.	Smoothing a
Gaussian Filter	Uses a Gaussian-weighted average, giving more importance to pixels near the center.	Image smoo
Median Filter	Replaces each pixel with the median value of the neighboring pixels.	Removing s

The **Average Filter** is simple but can blur edges. The **Gaussian Filter** provides smoother and more natural results because of its weighted averaging. The **Median Filter** is especially useful for impulse noise because it can remove extreme pixel values without significantly damaging edges.

---

4. Why is the Median Filter particularly effective for removing salt-and-pepper noise?

The Median Filter is effective for removing salt-and-pepper noise because it replaces each pixel with the **median value of its neighboring pixels** rather than calculating their average.

Salt-and-pepper noise contains extremely bright or extremely dark pixels. These abnormal values are usually outliers in a neighborhood. The median operation ignores the effect of these extreme values and selects a representative middle value.

As a result, the noise is removed while important image structures and edges are generally preserved better than with an average filter.

---

5. Explain the role of convolution kernels in spatial filtering.

A convolution kernel is a small matrix containing numerical values that defines how neighboring pixels should be processed during spatial filtering.

The kernel is moved across the image, and at each position, its values are multiplied by the corresponding image pixel values. The results are added together to produce the new pixel value.

Different kernels perform different operations. For example:

- An **averaging kernel** is used for image smoothing.
- A **Gaussian kernel** is used for smooth noise reduction.
- A **Sobel kernel** is used for edge detection.
- A **Laplacian kernel** is used for detecting rapid intensity changes and sharpening.

Thus, convolution kernels provide a mathematical method for controlling the effect of spatial filtering on an image.

---

6. What is the purpose of the Sobel and Laplacian operators in edge detection?

The **Sobel operator** and **Laplacian operator** are used to detect edges by identifying significant changes in pixel intensity.

The **Sobel operator** uses first-order derivatives and generally uses separate kernels to detect changes in the horizontal and vertical directions. It is useful for detecting edges while providing some smoothing because of its kernel structure.

The **Laplacian operator** uses a second-order derivative to detect rapid changes in intensity. It can detect edges in multiple directions but is more sensitive to noise.

Therefore, Sobel is commonly used when directional edge information is useful, while Laplacian is useful for detecting fine edges and image sharpening.

---

7. Why are filtering operations considered an essential preprocessing step in computer vision?

Filtering is an important preprocessing step because real-world images often contain **noise, unwanted variations, and other imperfections**.

Filtering can:

- Reduce image noise.
- Smooth unwanted variations.
- Improve image quality.
- Enhance important features and edges.
- Prepare images for segmentation and object detection.
- Improve the performance of feature extraction and machine learning models.

By producing a cleaner and more useful image, filtering helps later computer vision algorithms perform more reliably.

---

8. Discuss the trade-off between image smoothing and edge preservation during filtering.

Image smoothing reduces noise and unwanted variations, but excessive smoothing can also remove important edges and fine details.

For example, an average filter can effectively reduce noise but may blur object boundaries. A Gaussian filter provides smoother results but can still reduce some edge details. A median filter is often better at preserving edges while removing impulse noise.

Therefore, the filter and its parameters must be selected carefully. **More smoothing generally means less noise but greater risk of losing details, while less smoothing**

preserves details but may leave more noise.

The goal is to achieve a balance between **noise reduction and edge preservation**.

---

9. Mention four real-world applications where spatial filtering techniques are widely used.

Four real-world applications of spatial filtering are:

1. **Medical Imaging** – Used to reduce noise and enhance structures in X-ray, CT, MRI, and other medical images.
2. **Surveillance and Security** – Used for improving images and detecting edges or objects in surveillance systems.
3. **Photography and Image Enhancement** – Used for noise reduction, smoothing, sharpening, and improving image quality.
4. **Computer Vision and Autonomous Systems** – Used as preprocessing for object detection, image segmentation, and feature extraction.

---

10. Compare spatial domain filtering with frequency domain filtering in terms of implementation and practical applications.

Spatial Domain Filtering	Freq
Operates directly on image pixels.	Operates on the frequency representatior
Uses masks or convolution kernels.	Usually involves transforming the image u
Relatively simple to implement for small filters.	Can be more computationally involved be
Commonly used for smoothing, sharpening, and edge detection.	Useful for frequency-based noise remova
Examples include Average, Gaussian, Median, Sobel, and Laplacian filters.	Examples include frequency-domain Low

In spatial filtering, operations are performed directly on pixels and their neighborhoods. In frequency-domain filtering, the image is transformed into its frequency components, the desired frequencies are modified, and the image is transformed back.

In practice, spatial filtering is often preferred for simple local operations, while frequency-domain filtering is useful when the characteristics of image frequencies need to be analyzed or selectively modified.

Double-click (or enter) to edit

Double-click (or enter) to edit

Double-click (or enter) to edit